

09 - DQL: CONSULTANDO DADOS COM SQL

Banco de Dados • Data Query Language • SELECT • Projeto SmartCoffee

Abertura Inspiradora

“Guardar dados é importante. Saber fazer as perguntas certas aos dados é o que transforma um banco em informação.”

Apresentação

Nas aulas anteriores, modelamos entidades, definimos relacionamentos e cardinalidades e criamos estruturas utilizando SQL. Agora chegamos ao momento em que o banco começa a responder perguntas.

DQL — Data Query Language é a parte da SQL utilizada para consultar e recuperar dados armazenados no banco.

O principal comando da DQL é o **SELECT**.

Objetivos da Aula

- compreender o papel da DQL dentro da SQL;
- consultar uma tabela inteira ou apenas colunas específicas;
- filtrar dados com **WHERE** ;
- utilizar operadores relacionais e lógicos;
- pesquisar intervalos, listas, textos e valores nulos;
- ordenar e limitar resultados;
- eliminar duplicidades com **DISTINCT** ;
- criar colunas calculadas e aliases;
- utilizar funções de agregação;
- agrupar dados com **GROUP BY** e filtrar grupos com **HAVING** ;
- consultar entidades relacionadas utilizando **JOIN** ;
- interpretar resultados e transformar regras de negócio em consultas SQL.

1. Onde a DQL se encaixa?

Grupo	Finalidade	Exemplos
DDL	Definir estruturas	CREATE, ALTER, DROP
DML	Manipular dados	INSERT, UPDATE, DELETE
DQL	Consultar dados	SELECT

Observação: em algumas classificações, **SELECT** aparece dentro da DML. Nesta disciplina, vamos utilizar a separação didática DQL para destacar o estudo das consultas.

2. Nossa base: SmartCoffee

As demonstrações utilizarão as seguintes entidades:

CLIENTE

CATEGORIA

PRODUTO

PEDIDO

ITEM_PEDIDO

FORMA_PAGAMENTO

PAGAMENTO

CLIENTE 1 — N PEDIDO 1 — N ITEM_PEDIDO N — 1 PRODUTO N —
|
1 — N PAGAMENTO N — 1 FORMA_PAGAMENTO

Arquivo de apoio: acompanhe esta aula com o script

smartcoffee_dql_base.sql. Ele cria uma base pequena, consistente e já populada para todas as demonstrações.

3. O primeiro SELECT

A estrutura mais simples é:

```
SELECT coluna  
FROM tabela;
```

Para consultar todas as colunas:

EXEMPLO 1 — TODOS OS CLIENTES

```
SELECT *  
FROM cliente;
```

Leitura: “Selecione todas as colunas da tabela cliente.”

Selecionando apenas algumas colunas

EXEMPLO 2

```
SELECT nome, email  
FROM cliente;
```

Evitar **SELECT *** em consultas reais pode deixar a consulta mais clara e impedir a recuperação de dados desnecessários.

4. Alias com AS

Um **alias** é um apelido exibido no resultado da consulta.

```
SELECT nome AS cliente,  
       telefone AS contato  
FROM cliente;
```

Também podemos criar títulos para expressões calculadas.

```
SELECT nome,  
       preco,  
       preco * 0.90 AS preco_promocional  
FROM produto;
```

5. DISTINCT — eliminando repetições

Se quisermos visualizar apenas valores diferentes:

```
SELECT DISTINCT cidade  
FROM cliente;
```

Sem DISTINCT: Limeira poderia aparecer várias vezes.

Com DISTINCT: cada cidade aparece apenas uma vez.

6. WHERE — filtrando registros

O **WHERE** define quais linhas serão retornadas.

```
SELECT nome, preco
FROM produto
WHERE preco > 10;
```

Operadores de comparação

Operador	Significado
=	igual
<> ou !=	diferente
>	maior que
>=	maior ou igual
<	menor que
<=	menor ou igual

EXEMPLO 3 — PRODUTOS ATIVOS

```
SELECT nome, preco
FROM produto
WHERE ativo = TRUE;
```

EXEMPLO 4 — PEDIDOS ACIMA DE DETERMINADO VALOR

```
SELECT id_pedido, data_pedido, valor_total
FROM pedido
WHERE valor_total >= 25.00;
```

7. AND, OR e NOT

AND — todas as condições precisam ser verdadeiras

```
SELECT nome, preco
FROM produto
WHERE preco >= 8
      AND preco <= 20;
```

OR — pelo menos uma condição precisa ser verdadeira

```
SELECT nome, cidade
FROM cliente
WHERE cidade = 'Limeira'
      OR cidade = 'Americana';
```

NOT — negação

```
SELECT nome, cidade
FROM cliente
WHERE NOT cidade = 'Limeira';
```

Cuidado com AND e OR juntos. Use parênteses quando necessário para deixar a regra explícita.

```
SELECT nome, cidade, ativo
FROM cliente
WHERE ativo = TRUE
      AND (cidade = 'Limeira' OR cidade = 'Americana');
```

8. BETWEEN — pesquisando intervalos

BETWEEN inclui os limites inicial e final.

```
SELECT nome, preco
FROM produto
WHERE preco BETWEEN 8.00 AND 15.00;
```

Datas

```
SELECT id_pedido, data_pedido, valor_total
FROM pedido
WHERE data_pedido BETWEEN '2026-09-01 00:00:00'
                        AND '2026-09-30 23:59:59';
```

9. IN — várias possibilidades

Em vez de repetir muitos **OR** :

```
SELECT nome, cidade
FROM cliente
WHERE cidade IN ('Limeira', 'Americana', 'Campinas');
```

Para excluir uma lista:

```
SELECT nome, cidade
FROM cliente
WHERE cidade NOT IN ('Limeira', 'Americana');
```

10. LIKE — pesquisando textos

Coringa	Significado	Exemplo
%	zero ou vários caracteres	'Café%'
_	exatamente um caractere	'_aria'

COMEÇA COM “CAFÉ”

```
SELECT nome
FROM produto
WHERE nome LIKE 'Café%';
```

CONTÉM “CHOCOLATE”

```
SELECT nome
FROM produto
WHERE nome LIKE '%chocolate%';
```

TERMINA COM “SILVA”

```
SELECT nome
FROM cliente
WHERE nome LIKE '%Silva';
```

11. NULL — ausência de valor

NULL não significa zero nem texto vazio. Representa ausência de valor conhecido.

O teste correto é:

```
SELECT nome, telefone
FROM cliente
WHERE telefone IS NULL;
```

E para quem possui telefone:

```
SELECT nome, telefone
FROM cliente
WHERE telefone IS NOT NULL;
```

Erro comum: `telefone = NULL` não é a forma correta de testar nulos.

12. ORDER BY — ordenando resultados

ASC é crescente e **DESC** é decrescente.

```
SELECT nome, preco
FROM produto
ORDER BY preco ASC;
```

```
SELECT nome, preco
FROM produto
ORDER BY preco DESC;
```

Ordenação por mais de uma coluna

```
SELECT cidade, nome
FROM cliente
ORDER BY cidade ASC, nome ASC;
```

13. LIMIT — limitando a quantidade de linhas

CINCO PRODUTOS MAIS CAROS

```
SELECT nome, preco
FROM produto
ORDER BY preco DESC
LIMIT 5;
```

LIMIT com OFFSET

```
SELECT nome, preco
FROM produto
ORDER BY nome
LIMIT 5 OFFSET 5;
```

Esse recurso aparece com frequência em **paginação** de sistemas web.

14. Colunas calculadas

O banco pode realizar cálculos durante a consulta.

```
SELECT nome,
       preco,
       preco * 1.10 AS preco_com_reajuste
FROM produto;
```

Calculando subtotal de um item

```
SELECT id_item,
       quantidade,
       preco_unitario,
       quantidade * preco_unitario AS subtotal
FROM item_pedido;
```

15. Funções úteis em consultas

Textos

```
SELECT UPPER(nome) AS nome_maiusculo,  
       LOWER(email) AS email_minusculo  
FROM cliente;
```

```
SELECT CONCAT(nome, ' - ', cidade) AS cliente_cidade  
FROM cliente;
```

Números

```
SELECT nome,  
       preco,  
       ROUND(preco * 0.90, 2) AS preco_desconto  
FROM produto;
```

Datas

```
SELECT id_pedido,  
       data_pedido,  
       DATE(data_pedido) AS somente_data,  
       YEAR(data_pedido) AS ano,  
       MONTH(data_pedido) AS mes  
FROM pedido;
```

COALESCE — substituindo NULL na apresentação

```
SELECT nome,  
       COALESCE(telefone, 'Não informado') AS telefone  
FROM cliente;
```

16. Funções de agregação

Funções de agregação calculam um resultado a partir de várias linhas.

Função	Finalidade
COUNT ()	contar
SUM ()	somar
AVG ()	calcular média
MIN ()	menor valor
MAX ()	maior valor

QUANTOS CLIENTES EXISTEM?

```
SELECT COUNT(*) AS total_clientes  
FROM cliente;
```

PREÇO MÉDIO DOS PRODUTOS

```
SELECT AVG(preco) AS preco_medio  
FROM produto;
```

RESUMO DE PREÇOS

```
SELECT MIN(preco) AS menor_preco,  
       MAX(preco) AS maior_preco,  
       AVG(preco) AS preco_medio  
FROM produto;
```

TOTAL VENDIDO

```
SELECT SUM(valor_total) AS faturamento  
FROM pedido  
WHERE status = 'FINALIZADO';
```

17. GROUP BY — agrupando dados

Imagine a pergunta:

“Quantos clientes temos em cada cidade?”

```
SELECT cidade,  
       COUNT(*) AS quantidade_clientes  
FROM cliente  
GROUP BY cidade;
```

Quantidade de produtos por categoria

```
SELECT id_categoria,  
       COUNT(*) AS quantidade_produtos  
FROM produto  
GROUP BY id_categoria;
```

Ideia central: **GROUP BY** divide as linhas em grupos; a função de agregação calcula um resultado para cada grupo.

18. HAVING — filtrando grupos

WHERE filtra linhas **antes** do agrupamento. **HAVING** filtra os grupos **depois** do **GROUP BY**.

CIDADES COM PELO MENOS DOIS CLIENTES

```
SELECT cidade,  
       COUNT(*) AS quantidade_clientes  
FROM cliente  
GROUP BY cidade  
HAVING COUNT(*) >= 2;
```

Pergunta para os alunos: por que não usar **WHERE COUNT(*) >= 2**? Porque o **WHERE** acontece antes de os grupos e seus totais existirem.

19. A ordem de uma consulta

Uma consulta completa costuma ser escrita nesta ordem:

```
SELECT colunas
FROM tabela
WHERE condicao
GROUP BY colunas_de_agrupamento
HAVING condicao_do_grupo
ORDER BY colunas
LIMIT quantidade;
```

SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

A ordem lógica interna de processamento pelo SGBD é um assunto mais avançado; por enquanto, memorize a ordem de escrita e compreenda a função de cada cláusula.

20. JOIN — consultando entidades relacionadas

Na Aula 07, usamos chaves estrangeiras para representar relacionamentos. Agora vamos utilizar essas FKs para recuperar informações relacionadas.

CLIENTE 1 — N PEDIDO
PK FK
id_cliente ← id_cliente

INNER JOIN

Retorna registros que possuem correspondência entre as tabelas.

PEDIDOS COM O NOME DO CLIENTE

```
SELECT p.id_pedido,
       p.data_pedido,
       c.nome AS cliente,
       p.valor_total
FROM pedido AS p
INNER JOIN cliente AS c
      ON c.id_cliente = p.id_cliente;
```

Leia o ON: “relacione o cliente cujo **id_cliente** é igual ao **id_cliente** gravado no pedido”.

Aliases de tabela

p e **c** são aliases que deixam consultas com várias tabelas mais curtas e legíveis.

21. LEFT JOIN

O **LEFT JOIN** mantém **todos os registros da tabela da esquerda**, mesmo quando não existe correspondência na tabela da direita.

TODOS OS CLIENTES, INCLUSIVE QUEM NUNCA COMPROU

```
SELECT c.nome,  
       p.id_pedido,  
       p.data_pedido  
FROM cliente AS c  
LEFT JOIN pedido AS p  
      ON p.id_cliente = c.id_cliente  
ORDER BY c.nome;
```

Descobrimos clientes sem pedidos

```
SELECT c.id_cliente, c.nome  
FROM cliente AS c  
LEFT JOIN pedido AS p  
      ON p.id_cliente = c.id_cliente  
WHERE p.id_pedido IS NULL;
```

Aplicação real: essa consulta poderia ajudar uma campanha de relacionamento com clientes cadastrados que ainda não compraram.

22. JOIN com três tabelas

Podemos percorrer relacionamentos sucessivos.

ITENS DE PEDIDO COM NOME DO PRODUTO

```
SELECT p.id_pedido,  
       pr.nome AS produto,  
       ip.quantidade,  
       ip.preco_unitario,  
       ip.quantidade * ip.preco_unitario AS subtotal  
FROM pedido AS p  
INNER JOIN item_pedido AS ip  
      ON ip.id_pedido = p.id_pedido  
INNER JOIN produto AS pr  
      ON pr.id_produto = ip.id_produto  
ORDER BY p.id_pedido;
```

23. JOIN com categoria

PRODUTO E NOME DE SUA CATEGORIA

```
SELECT pr.nome AS produto,  
       c.nome AS categoria,  
       pr.preco  
FROM produto AS pr  
INNER JOIN categoria AS c  
      ON c.id_categoria = pr.id_categoria  
ORDER BY c.nome, pr.nome;
```

24. Agregação + JOIN

Quantidade de produtos por categoria

```
SELECT c.nome AS categoria,  
       COUNT(pr.id_produto) AS quantidade_produtos  
FROM categoria AS c  
LEFT JOIN produto AS pr  
      ON pr.id_categoria = c.id_categoria  
GROUP BY c.id_categoria, c.nome  
ORDER BY quantidade_produtos DESC;
```

Total comprado por cliente

```
SELECT c.nome AS cliente,  
       COUNT(p.id_pedido) AS quantidade_pedidos,  
       SUM(p.valor_total) AS total_comprado  
FROM cliente AS c  
INNER JOIN pedido AS p  
      ON p.id_cliente = c.id_cliente  
WHERE p.status = 'FINALIZADO'  
GROUP BY c.id_cliente, c.nome  
ORDER BY total_comprado DESC;
```

25. Relatório completo do pedido

Agora combinamos várias entidades em uma consulta de relatório.

```
SELECT p.id_pedido,
       DATE_FORMAT(p.data_pedido, '%d/%m/%Y %H:%i') AS data,
       c.nome AS cliente,
       pr.nome AS produto,
       ip.quantidade,
       ip.preco_unitario,
       ip.quantidade * ip.preco_unitario AS subtotal
FROM pedido AS p
INNER JOIN cliente AS c
      ON c.id_cliente = p.id_cliente
INNER JOIN item_pedido AS ip
      ON ip.id_pedido = p.id_pedido
INNER JOIN produto AS pr
      ON pr.id_produto = ip.id_produto
ORDER BY p.id_pedido, pr.nome;
```

26. CASE — criando classificações no resultado

CASE permite construir uma classificação baseada em regras.

```
SELECT nome,
       preco,
       CASE
         WHEN preco < 8 THEN 'Econômico'
         WHEN preco <= 15 THEN 'Intermediário'
         ELSE 'Premium'
       END AS faixa_preco
FROM produto
ORDER BY preco;
```

Importante: isso não altera os dados armazenados; apenas produz uma informação calculada no resultado.

27. Subconsulta — introdução opcional

Uma subconsulta é uma consulta dentro de outra consulta. É um conteúdo de aprofundamento.

PRODUTOS ACIMA DO PREÇO MÉDIO

```
SELECT nome, preco
FROM produto
WHERE preco > (
    SELECT AVG(preco)
    FROM produto
)
ORDER BY preco DESC;
```

Primeiro o banco calcula a média; depois utiliza esse valor como filtro da consulta externa.

28. Perguntas de negócio virando SQL

Pergunta	Recursos SQL prováveis
Quais produtos custam mais de R\$ 15?	SELECT + WHERE
Quais são os 3 produtos mais caros?	ORDER BY + LIMIT
Quantos pedidos cada cliente realizou?	JOIN + COUNT + GROUP BY
Quais clientes nunca fizeram pedido?	LEFT JOIN + IS NULL
Qual o faturamento dos pedidos finalizados?	SUM + WHERE
Quais categorias possuem mais de 2 produtos?	JOIN + GROUP BY + HAVING
Quais produtos estão acima da média de preço?	Subconsulta + AVG

29. Estratégia para construir uma consulta

1. Transforme o problema em uma pergunta clara.
2. Descubra quais tabelas possuem os dados.
3. Escolha as colunas que precisam aparecer.
4. Verifique se será necessário JOIN.
5. Defina filtros com WHERE.
6. Verifique se existe cálculo ou agrupamento.
7. Organize a apresentação com ORDER BY.
8. Execute e confira se o resultado faz sentido.

Programar uma consulta não é apenas acertar a sintaxe. É verificar se a resposta do banco corresponde à pergunta de negócio.

30. Erros comuns em DQL

- esquecer o nome da tabela após **FROM** ;
- usar texto sem aspas: **cidade = Limeira** ;
- usar **= NULL** em vez de **IS NULL** ;
- misturar **AND** e **OR** sem parênteses;
- esquecer a condição **ON** de um JOIN;
- relacionar colunas erradas no **ON** ;
- usar **WHERE** para filtrar uma função agregada em vez de **HAVING** ;
- selecionar colunas não agrupadas junto de agregações sem compreender o resultado;
- não usar aliases quando duas tabelas possuem colunas com o mesmo nome;
- considerar correto um resultado apenas porque a consulta executou sem erro.

31. Demonstração guiada — evolução de uma consulta

Começaremos com uma pergunta simples e evoluiremos passo a passo.

Passo 1 — visualizar produtos

```
SELECT * FROM produto;
```

Passo 2 — selecionar somente o necessário

```
SELECT nome, preco FROM produto;
```

Passo 3 — filtrar

```
SELECT nome, preco  
FROM produto  
WHERE preco >= 10;
```

Passo 4 — ordenar

```
SELECT nome, preco
FROM produto
WHERE preco >= 10
ORDER BY preco DESC;
```

Passo 5 — mostrar a categoria

```
SELECT pr.nome AS produto,
       c.nome AS categoria,
       pr.preco
FROM produto AS pr
INNER JOIN categoria AS c
      ON c.id_categoria = pr.id_categoria
WHERE pr.preco >= 10
ORDER BY pr.preco DESC;
```

Passo 6 — transformar em pergunta gerencial

```
SELECT c.nome AS categoria,
       COUNT(pr.id_produto) AS produtos,
       ROUND(AVG(pr.preco), 2) AS preco_medio
FROM categoria AS c
INNER JOIN produto AS pr
      ON pr.id_categoria = c.id_categoria
GROUP BY c.id_categoria, c.nome
ORDER BY preco_medio DESC;
```

32. Atividade Prática — Analista de Dados da SmartCoffee

Situação-problema: a gestão da SmartCoffee precisa entender melhor clientes, produtos e vendas. Sua equipe foi chamada para produzir consultas que respondam às perguntas abaixo. Para cada item, escreva e execute a consulta SQL e confira o resultado.

Parte A — Aquecimento Básico

1. Liste todos os clientes cadastrados.
2. Exiba apenas nome, cidade e e-mail dos clientes.
3. Liste os nomes das cidades sem repetir valores.

4. Liste todos os produtos em ordem crescente de preço.

5. Mostre apenas os 5 produtos mais caros.

Parte B — Filtros Básico/Intermediário

6. Liste os produtos com preço entre R\$ 8,00 e R\$ 15,00.

7. Liste os clientes das cidades Limeira ou Americana.

8. Localize os produtos cujo nome contém a palavra “Café”.

9. Liste os clientes que não informaram telefone.

10. Mostre os pedidos com status FINALIZADO e valor total acima de R\$ 20,00, do maior para o menor valor.

Parte C — Cálculos e agrupamentos Intermediário

11. Informe quantos produtos estão cadastrados.

12. Mostre menor preço, maior preço e preço médio dos produtos.

13. Informe quantos clientes existem em cada cidade.

14. Mostre somente as cidades que possuem dois ou mais clientes.

15. Calcule o faturamento total considerando apenas pedidos FINALIZADOS.

Parte D — Relacionamentos Intermediário

16. Liste cada pedido exibindo: id do pedido, data, nome do cliente, status e valor total.

17. Liste cada produto acompanhado do nome de sua categoria.

18. Gere um relatório dos itens vendidos exibindo: pedido, produto, quantidade, preço unitário e subtotal.

19. Mostre todos os clientes, inclusive aqueles que nunca fizeram pedidos.

20. Liste apenas os clientes que nunca fizeram pedidos.

Parte E — Desafio Gerencial Avançado

21. Informe quantos pedidos FINALIZADOS cada cliente realizou e quanto cada cliente gastou. Ordene do maior gasto para o menor.

22. Mostre a quantidade de produtos e o preço médio de cada categoria.

23. Descubra quais produtos possuem preço superior ao preço médio de todos os produtos.

24. Classifique os produtos como “Econômico”, “Intermediário” ou “Premium”, utilizando faixas de preço definidas por você.

25. Crie uma consulta autoral que responda a uma pergunta útil para a gestão da SmartCoffee. Escreva primeiro a pergunta de negócio e depois a consulta SQL.

Entrega sugerida

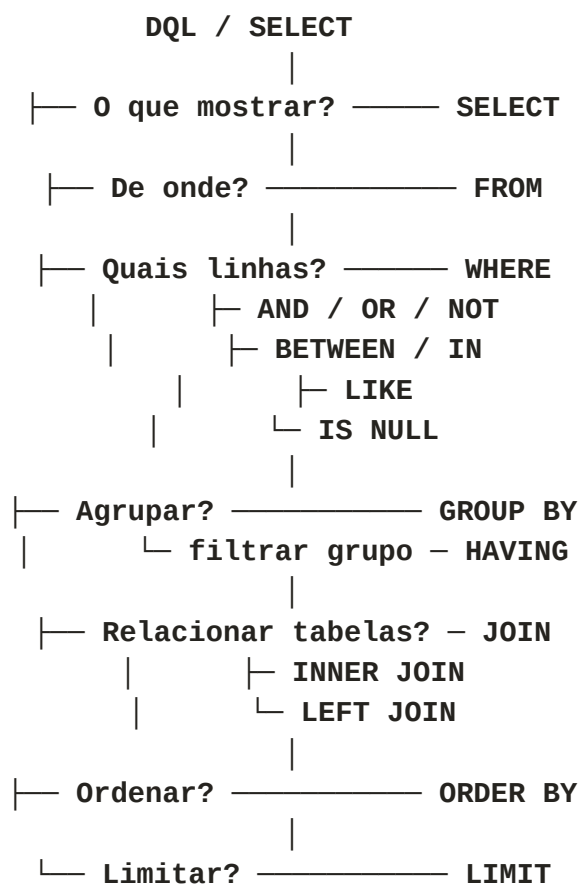
- arquivo `.sql` com as consultas numeradas de 1 a 25;
- comentário antes de cada consulta explicando a pergunta respondida;
- as consultas devem executar sem erro;
- na questão 25, justificar em uma frase por que a informação seria útil ao negócio.

33. Checklist do aluno

- ☐ Sei explicar o que é DQL.
- ☐ Consigo selecionar colunas específicas.
- ☐ Sei utilizar WHERE.
- ☐ Consigo combinar condições com AND e OR.
- ☐ Sei utilizar BETWEEN, IN e LIKE.

- ☐ Sei testar NULL corretamente.
- ☐ Consigo ordenar e limitar resultados.
- ☐ Sei utilizar COUNT, SUM, AVG, MIN e MAX.
- ☐ Entendo GROUP BY e HAVING.
- ☐ Consigo relacionar duas tabelas com INNER JOIN.
- ☐ Entendo para que serve LEFT JOIN.
- ☐ Consigo transformar uma pergunta de negócio em uma consulta.

34. Mapa Mental



Encerramento

Uma boa consulta SQL começa antes do SELECT: começa sabendo exatamente qual pergunta queremos responder.

